

WHITEPAPER

Managing AI Agents at Scale: The Agent Development Lifecycle

Most enterprises are experimenting with AI agents. Almost none are managing them in production at scale. The ADLC is a structured framework for changing that.

Published by Solace

Executive Summary

Most enterprises are experimenting with AI agents, but very few are running them in production, let alone at scale. The gap between those two states is where most agentic initiatives are stalling, and why organizations aren't realizing the ROI they hope for.

Unlike traditional software applications, AI agents reason, plan, and act—capabilities that create new operational possibilities but introduce significant new challenges. Most organizations today are deploying these agents without the structured processes that production-grade systems and high-performing teams require.

This whitepaper introduces the agent development lifecycle (ADLC): a six-stage framework for conceiving, building, deploying, governing, and continuously improving AI agents in the enterprise. Drawing on proven practices from software engineering (namely software development lifecycle (SDLC), distributed systems design, and organizational management) ADLC provides a coherent, repeatable and scalable approach to agent development.

ADLC also maps to the lifecycle of an employee - from hiring through onboarding, coaching, supervision, teamwork, and continuous improvement. That's because the underlying challenge is the same: transforming individual capability into organizational output.

Organizations that adopt ADLC gain more than a development process. They gain the governance infrastructure needed to build trust in agentic systems, the architectural discipline it takes to scale beyond individual agents, and the operational rigor required to deliver real business value from agentic AI.

Introduction

Most enterprises are building and experimenting with AI agents, but running them reliably at scale is a different discipline entirely.

As organizations begin treating agents as workers within their operations—participants in workflows, decision-makers within defined boundaries, and collaborators across functional domains—a question arises: are these systems being built with the same care and structure applied to production-grade software and high-performing human teams?

The early evidence suggests the answer is mostly no, and the cost is already visible. Gartner forecasts that more than 40% of agentic AI projects will be canceled by the end of 2027, victims of rising costs, unproven business value, and inadequate risk controls.

MIT's 2025 study of enterprise GenAI adoption landed even harder: 95% of pilots have produced no measurable return on an estimated \$30 to \$40 billion in spend, with most stuck at the experimental stage. The researchers' diagnosis was not that the models are failing; it was that the organizations deploying them lack the integration, feedback discipline, and operational structure required to move these systems into production.

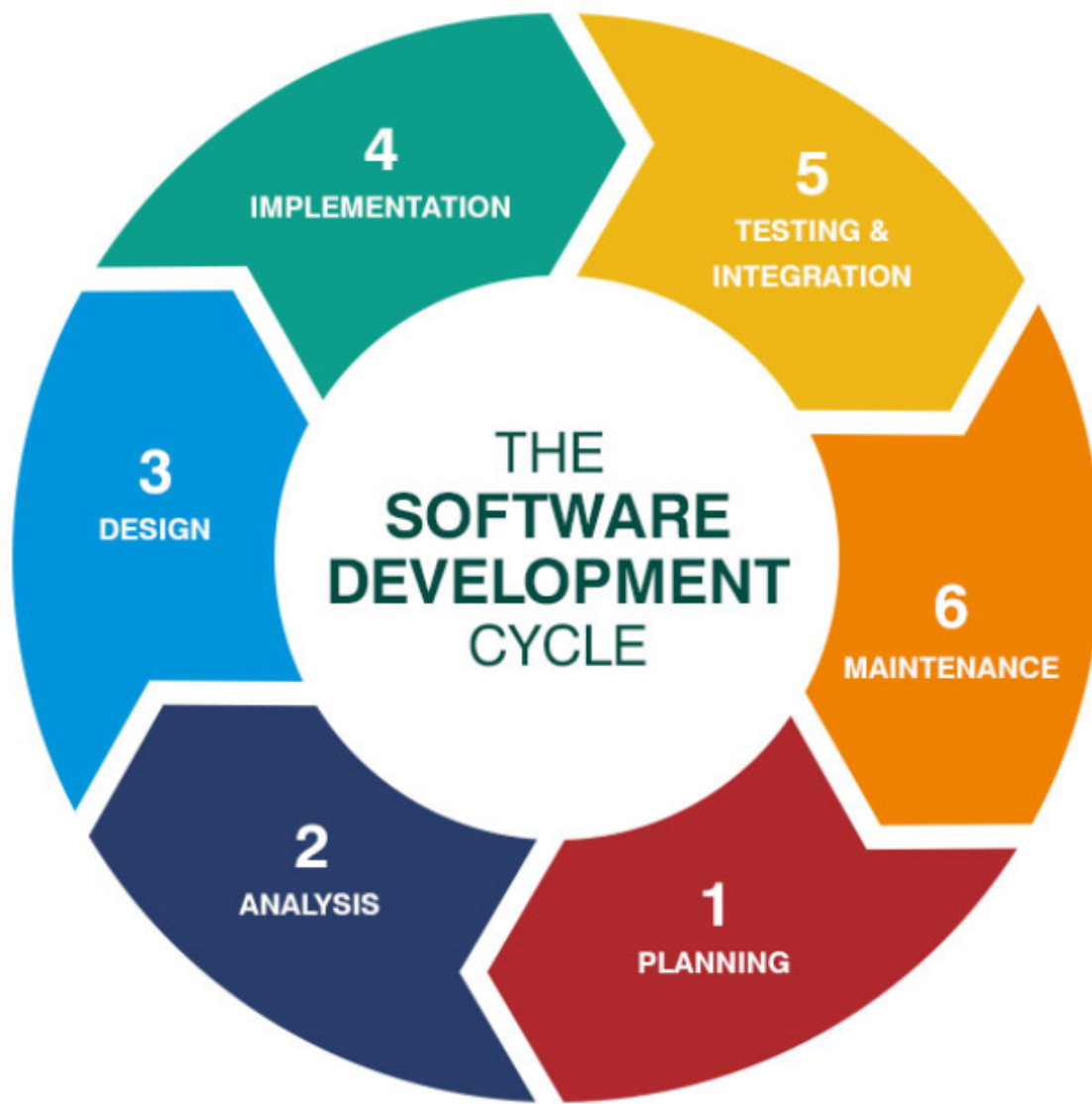
The minority that do succeed do so because they customize agents for specific processes, integrate them tightly with existing operations, and build continuous improvement into the system itself. Simply put, this is a process problem.

The software industry has long understood that reliable systems require a structured, repeatable way of building them. That insight is the foundation of the software development lifecycle (SDLC). In this paper we'll make the case that agentic systems demand the same discipline, but with their own nuances, and an adapted approach called the *agent* development lifecycle (ADLC).

Learning From the Past: SDLC

Without a disciplined approach to how software is planned, built, tested, and maintained, projects drift, quality erodes, and systems become impossible to scale. Software built without this structure tends to be fast at first and brittle forever. This truth gave rise to the software development lifecycle (SDLC), a framework that codified methodology as a core ingredient of reliable software delivery.

The SDLC did not merely organize developers—it transformed how teams approached accountability, quality, and continuity. It proved that the discipline applied to building applications matters as much as the talent brought to it. Most SDLC adaptations move through six to seven stages: planning, analysis, design, implementation, testing and integration, and maintenance. What makes the SDLC powerful is not any individual stage but the discipline of moving through all of them deliberately and repeatedly.



Key Principle: The discipline you bring to building a system matters as much as the talent you bring to it. This truth applies equally to employees, software and to the agents that increasingly power enterprise operations.

Introducing the Agent Development Lifecycle (ADLC)

Agents are software, so it shouldn't be surprising that the agent development lifecycle (ADLC) applies SDLC's philosophy of structured, repeatable process to the development and management of AI agents.

Extending Proven SDLC Techniques to AI Agents

Just as the SDLC defines how software moves from concept to production and through ongoing maintenance, the ADLC defines how an agent is conceived, built, deployed, and continuously improved within an enterprise environment.

However, AI agents are not conventional software. Since they are based on probabilistic language models, large or small language models (LLM/SLM), there are some subtle but fundamental differences in the way agents are built, evaluated, and governed.

API-first development introduced its own set of practices that did not exist in traditional software delivery: versioning contracts, rate limiting, schema governance, and consumer-driven design. Agents represent a similar inflection point, where the medium is familiar, but the rules of engagement are new.

Lessons from Human Resources Practices

The ADLC also incorporates lessons learned from the lifecycle of an employee. When an organization brings on a new team member, there is a deliberate process: their role is defined, they're given access to the systems they need, trained, and integrated into a team. Once they're empowered and put to work, a manager oversees their activity...monitoring and developing their performance over time. As autonomous actors operating within a system of governance, AI agents deserve and require that same treatment.

Having an ADLC is necessary to get real value out of agentic AI, but a process requires tools to support it. We will now present the phases of an ADLC in more detail and break down the functionality you should look for in an agentic platform to make sure you can implement all phases of ADLC.



The Six Stages of the ADLC

Stage 1: Hiring – Define the Role Before You Build the Agent

Before writing a single line of agent configuration, the organization must define what the agent is for. This means establishing its responsibilities, its scope of authority, and what success looks like in measurable terms. A frequent failure in agentic deployments is building agents without clear role definitions - the AI equivalent of hiring a new employee without a job description.

This stage encompasses the creation of a well-specified system prompt that establishes the agent's role, skills, behavioral parameters, and boundaries. Skills are reusable, domain-specific capability modules that provide agents with procedural guidance relevant to their role. This stage also includes the definition of guardrails: explicit constraints on what the agent can and cannot do, preventing scope creep, unsafe actions, and unpredictable behavior in production.

Operationally, this stage involves the use of no-code or pro-code agent-builder tools and prompt-engineering. Organizations should treat this process as an organizational decision, not merely a technical one - determining the role the agent will play, the outcomes it will be held accountable for, and the boundaries within which it is authorized to act.

- **Platform Requirements:** No-code AI assisted and pro-code agent builders to define the agent's system prompt, instructions, skills, and guardrails. This defines the foundational set of rules, personas, and constraints given to an AI agent before it interacts with a user. Such capabilities allow fast prototyping and development of agents without needing to be an expert software developer.
- **Enterprise Consideration:** An agent without a clear role definition behaves like an employee without a job description: duplicating work, overstepping boundaries, or stalling on decisions. Role clarity at this stage is the single most important determinant of agent reliability downstream.

Example: A luxury-goods retailer is looking to lift average sale value per customer visit by combining specialist agents for inventory, taste profile, and purchase history at the moment of in-store interaction. They will need to build agents for inventory checking, customer taste profile assessment, purchase history assessment, which all feed into a real-time product recommendation agent.

Stage 2: Onboarding – Give Agents the Access They Need

Once the role is defined, the agent must be connected to the systems and data sources required to perform it. This onboarding stage is the direct analog to provisioning a new employee with the tools, credentials, and access rights required on their first day - email, collaboration platforms, data access, code repositories, and enterprise application logins.

For AI agents, onboarding encompasses connectivity to enterprise databases and data warehouses via SQL, enterprise applications via APIs, Model Context Protocol (MCP) servers, and third-party agents reached through Agent-to-Agent (A2A) protocols.

Then there is triggering the agent to do your work. Do you do that via a chat, an event, an API call, an email, or from another agent via A2A?

Getting onboarding right is about more than providing access. It is about providing the right access. An agent with an overly broad set of tools and integrations will be overloaded with context and behave unpredictably. An agent that lacks the right connections will lack the real-time data to make effective decisions.

The well-established principle of “least privilege” applies here. Role-based access controls (RBAC), single sign-on (SSO), and delegated access (where an agent operates on behalf of a specific user and inherits that user's permissions rather than acting with a generic service account) should govern every agent's integration surface, ensuring each agent receives only the permissions appropriate to its defined function and the identity on whose behalf it is acting.

- **Platform Requirements:** built-in real-time data connectors, extensibility to build your own data connectors, pre-built agent tools, extensibility to build and plug-in custom tools.

Example: A large Middle-Eastern bank had AI agents proliferating across business units on different frameworks—siloes, ungoverned, and not well orchestrated. Standardizing onboarding through a centralized platform with SSO, event-driven workflows, and governed access replaced fragmented point solutions with one governed environment that scales from ten agents to thousands without rearchitecting.

Stage 3: Coaching – Validate Competence

The coaching stage is where investment is made in evaluations to confirm that an agent performs as expected across the range of tasks it will encounter in production. Just as an organization would not place a new hire in front of its most demanding clients before confirming they perform well in the role, agents should not reach production without structured competence validation.

Evaluation frameworks (evals) are critical infrastructure at this stage. Evals should test not merely whether an agent produces the correct output, but whether it does so consistently, within the right boundaries, and without unintended side effects. Offline evals test agent behavior against a defined test harness before deployment. Online evals monitor agent performance against real-world outputs in production during and after deployment, providing the signal required for continuous improvement. This is typically achieved with techniques known as “LLM as a Judge”.

Coaching is not a one-time gate before initial deployment. It is a recurring process that feeds back into agent improvement throughout the ADLC.

Platform Requirements

- **Offline Evals** that enable the creation of test suites with expected outputs and tool usage patterns. These test suites can run locally or against remote deployments, with support for multiple evaluation methods, and the ability to run these evaluations periodically to detect degradation or improvements in agent performance.
- **Visualizer** provides an intuitive visual view of agent execution workflows to aid in understanding, enhancing & debugging agent behaviors.
- **Telemetry** to aid in agent health monitoring through the ability to route rich agentic insights to existing observability platforms (Datadog, Splunk, Dynatrace, etc.)

Example: A leading pure-play coffee company is cutting commercial customer-service order investigation time from hours to minutes. Different agents search across multiple back-end order management systems. The performance of the system in successfully finding lost and stalled orders is evaluated using a test harness built from records of previously problematic orders.

Stage 4: Supervision – Maintain Oversight

Autonomy is the goal of agentic systems, but unsupervised autonomy in LLM-powered workflows introduces material operational risk. The supervision stage establishes the appropriate level of human oversight for the tasks agents are performing, calibrated to the stakes and uncertainty of those tasks.

Human-in-the-loop (HITL) mechanisms must be treated as architectural components, not afterthoughts. For low-stakes, well-understood tasks, supervision may take the form of monitoring dashboards and automated anomaly detection. For high-stakes decisions – converting emails into orders, processing insurance claims, issuing purchase orders, generating customer-facing responses to sensitive inquiries - explicit checkpoints must be designed into the workflow, requiring human review and approval before the agent proceeds.

Agents operating in enterprise environments should also be configured to recognize low-confidence or high-risk conditions and escalate proactively to human oversight. This dynamic escalation capability is what makes enterprise deployment at scale viable: it enables organizations to extend automation progressively as trust is established, rather than requiring a binary choice between full automation and full manual review.

- **Platform Requirements:** Agentic Human-in-the-Loop (HITL) where an autonomous AI agent manages a complex workflow but strategically "pauses" to request human intervention for high-stakes decisions, edge cases, or quality assurance.
- **Enterprise Consideration:** Supervision is not a constraint on agent capability. It is the mechanism by which trust in agent capability is built - incrementally, verifiably, and in proportion to demonstrated performance and risk profile.

Example: A commercial trucking company receives thousands of shipping requests by email each month; manual order entry was taking time, producing errors, lost revenue, and poor fleet utilization. A multi-agent workflow now extracts, enriches, and routes orders automatically - unstructured email to structured data in seconds, with a human-in-the-loop review step at the supervision checkpoint. Over time, that HITL threshold can be progressively relaxed as the system earns trust.

Stage 5: Teamwork – Integrate Agents into an Ecosystem

In most enterprise deployments, an agent offers more value when it is one actor within a larger mesh of agents. The teamwork stage addresses how that agent is designed to function within the broader ecosystem, how it receives tasks, hands off results, and is orchestrated to collaborate with peer agents across functional domains.

Multi-agent architectures require hierarchical design: clear reporting structures, well-defined escalation paths, and governance mechanisms that keep the system secure and coherent as it scales.

Orchestrator agents coordinate specialist agents, routing tasks based on context, data connections, and capability. There are two types of complementary coordination patterns:

- **Deterministic “Workflows”:** the sequence of steps is known in advance and repeatability is paramount.
- **Dynamic “Orchestration”:** an orchestrator agent intelligently determines the plan at runtime based on the task, the available specialist agents, and the current state of the system.

Event-triggered and agentic automation use cases usually revolve around deterministic workflows, while chat-based agents lean towards dynamic orchestration. An agentic platform that can satisfy both fixed

workflows and dynamic orchestration is therefore able to address a broader set of use cases in a more uniform way for design, deployment, monitoring, and security.

Designing multi-agent systems is fundamentally similar to designing an organization: the agents you build should be able to function as a high-performing team, not just a collection of capable individuals.

Platform Requirements:

- **Deterministic Workflows:** configurable instruction template sequences that agents execute step-by-step, but with each step intelligently informed by AI. This foundational capability allows enterprises to implement reliable, repeatable, multi-step AI processes. These are most useful for process automation.
- **Dynamic Orchestration:** the process by which a central controller or "orchestrator" agent intelligently routes tasks, selects tools, and manages the hand-offs between specialized sub-agents in real-time, adapting the workflow on the fly based on the evolving context of the problem. This is most useful in conversational use cases.

Example: A global airline was routinely over-fueling its aircraft, driving up both costs and emissions, and reducing capacity for carrying high-value freight. A deterministic multi-agent workflow now coordinates specialist agents for flight plan, aircraft type, fuel type, and weather to produce an optimal fuel recommendation for each flight. It is a textbook teamwork stage: specialists operating as a team rather than as a pile of point scripts.

Stage 6: Improvement — Monitor, Measure, and Iterate

The final stage of ADLC is the feedback loop that keeps the system improving. As an analogy, when you hire an employee and they start working, you review their performance periodically and give them feedback, so their performance improves. This is how junior developers become your most knowledgeable and capable senior developers or architect leaders of the future. The same care in monitoring, feedback and improvement should be applied to agents.

This stage encompasses real-time observability of agent interactions and decision pathways, ongoing evaluation pipelines that detect performance drift before it causes damage, and structured performance reporting that surfaces trends over time. The goal is not merely to catch failures but to build a continuous feedback loop that makes agents measurably better over time, as you would do with your human employees.

Operationally, this requires investment in observability tooling: agent execution visualization, telemetry instrumentation, logging and online eval pipelines that run against live outputs. This is what you should be thinking about in order to actually effect continuous improvement:

- Gather detailed information about agent execution, decisions and tool calls across workflows.
- Have LLMs review to compare their result to the original mandate – known as online evals using LLM as a Judge.

- Critically, humans need to review to provide feedback (e.g. thumbs up/down) on whether the agents and workflows did a good job and importantly "why" it is good or bad - just like an employee performance review.

This feedback needs to be the basis for improving the agent's system prompts, skills, and tool definitions. And your ideal system should have agentic support to analyze all this data and propose better prompts, skills, tool definitions, so your agentic processes improve over time.

Humans act as reviewers and teachers, while agentic intelligence proposes improvements you can act on.

This mirrors the performance management process for human workers: structured feedback, regular assessment, and a defined path for development. Deployment is not the end of the lifecycle. It is day one of the ongoing career.

Platform requirements:

- **Visualizer:** a real-time graphical interface for tracing agent interactions, process flows, and decision pathways. You can see exactly what your agents are doing and why.
- **Online Evals:** detect performance drift or emerging failure modes before they cause damage.
- **Performance Telemetry:** surfaces performance trends over time, giving you the data to make informed decisions about when and how to improve.

Example: A large maritime port authority screens every vessel's Pre-Arrival Notification against an evolving high-risk list, a task manual teams could no longer keep pace with. An event-driven agent workflow now validates and screens each arrival event against standard operating procedures in real time. Threat response has dropped from hours to minutes, and the quality of threat assessment has improved based on feedback from the results of vessel screenings.

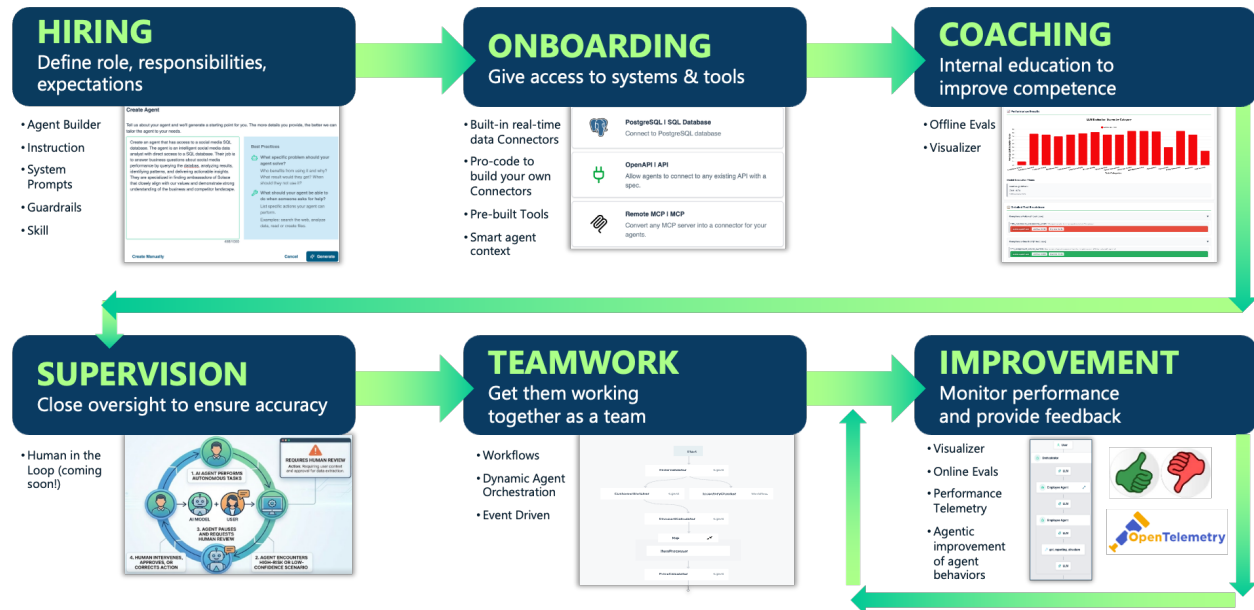
The ADLC is more than an analogy to employee onboarding. It is a direct application of the principles that make the full employee lifecycle effective, from hiring through ongoing improvement. When organizations invest in structured onboarding, clear role definitions, and ongoing development programs, they see measurably better performance, lower attrition, and faster time to productivity. The same returns are available in agentic systems, but only if the investment is made.

How Agentic AI Frameworks Support the ADLC

The ADLC is a process framework, not a product, and processes only create value when they can be operationalized. An effective agentic AI framework (like our own Solace Agent Mesh) must let enterprises run the ADLC from end to end and deploy agents to production.

It must provide the tooling to build agents, the event-driven communications backbone to orchestrate and govern them in production, and the observability and evaluation tools it takes to continuously improve them. Critically, it should do all of this while remaining open and vendor-neutral: any cloud, any large language model, any agent.

This image maps the six stages of the ADLC to the capabilities it takes to operationalize each one, and the descriptions below explain each one in greater detail. Consider this a checklist of things to look for when evaluating agentic AI frameworks.



Hiring — Define the Role

- **Agent Definition and Authoring:** The framework should enable teams to define agent roles, behaviors, and constraints so agents are purpose-built for specific tasks and operate within clearly defined boundaries. This is typically achieved through a combination of AI-assisted development tools, including no-code and pro-code interfaces for authoring system prompts, skills, and guardrails.

Onboarding — Provision Access

- **Enterprise Data Connectivity:** The framework should enable agents to securely connect to enterprise systems, data sources, and external services so they can participate in real workflows. This is typically achieved through SQL integration with databases, API integration with enterprise applications, support for agentic protocols like MCP and A2A.
- **Access Control and Identity:** The framework should ensure agents operate with the appropriate permissions and identity so their actions are secure, auditable, and aligned with enterprise policies. This is typically achieved through role-based access control (RBAC), single sign-on (SSO), and delegated identity models where agents act on behalf of specific users.
- **Tooling:** The framework should enable agents to use pre-built and custom tools, small programs or APIs that the LLM is given permission to trigger when it realizes it can't solve a problem with text alone. Tools may be used for things like analyze data, analyzing multi-media content, visualizing data, and managing artifacts.

- **Context Management:** The framework should be able to prevent agents and the underlying LLM from being overwhelmed by irrelevant or excessive data, which can result in poor performance, high-token consumption, and hallucinations. This is typically achieved by mechanisms for doing calculations and managing artifacts locally, controlling what information enters and persists within the agent's context.

Coaching — Validate Competence

- **Offline Evaluation:** The framework should enable teams to systematically validate agent behavior before and after deployment so agents perform reliably, consistently, and within defined boundaries. This is typically achieved through offline and online evaluation pipelines, including test harnesses, scoring mechanisms, and techniques such as LLM-as-a-judge.
- **Execution Visibility:** The framework should provide visibility into how agents collaboratively execute tasks so teams can understand, debug, and build trust in agent behavior. This is typically achieved through visualizations of workflows, tool usage, and decision paths.

Supervision — Maintain Oversight

- **Human-in-the-Loop Control:** The framework should enable appropriate levels of human oversight so agents can operate autonomously while still managing risk in high-stakes or uncertain scenarios. This is typically achieved through configurable checkpoints, approval workflows, and escalation mechanisms based on confidence or context.

Teamwork — Integrate the Ecosystem

- **Deterministic Workflows:** The framework should support structured, repeatable multi-step processes so agents can execute reliable, high-confidence operations. This is typically achieved through predefined workflows where each step is guided by AI but follows a known sequence.
- **Dynamic Orchestration:** The framework should enable agents to coordinate and collaborate dynamically so they can adapt to changing context and complex tasks at runtime. This is typically achieved through orchestrator agents that route tasks, select tools, and manage interactions between specialized agents.
- **Event-Driven Communication:** The framework should enable agents to run asynchronously and in parallel, so systems remain decoupled, resilient, and scalable as they grow. This is typically achieved through event-driven messaging patterns rather than synchronous, tightly coupled API calls.

Improvement – Monitor and Iterate

- **Continuous Evaluation:** The framework should enable ongoing assessment of agent performance so degradation, drift, and emerging failure modes are detected early. This is typically achieved through online evaluations that analyze live outputs and incorporate human feedback.

Observability, Telemetry and Visualization: The framework should provide deep visibility into agent activity so teams can monitor performance, detect issues, and understand system behavior over time. This is typically achieved through logging, tracing, and telemetry integrated with existing observability platforms.

- **Feedback-Driven Improvement:** The framework should enable continuous refinement of agents so performance improves over time based on real-world usage. This is typically achieved by combining evaluation data, human feedback, and system telemetry to generate recommendations for improving agent prompts, skills, and tool definitions.

The common thread is that an agentic AI framework should treat development, deployment, and improvement over time as part of a single discipline, not a collection of disparate tools. That is what a complete agentic lifecycle platform looks like in practice; that's what turns a sound framework into durable enterprise value; and that's precisely what we've built Solace Agent Mesh to do.

Conclusion

Most enterprises will deploy experimental AI agents. Only a few will manage to deploy them in production at scale. The ADLC, and the platforms purpose-built to operationalize it, is how the second group gets there.

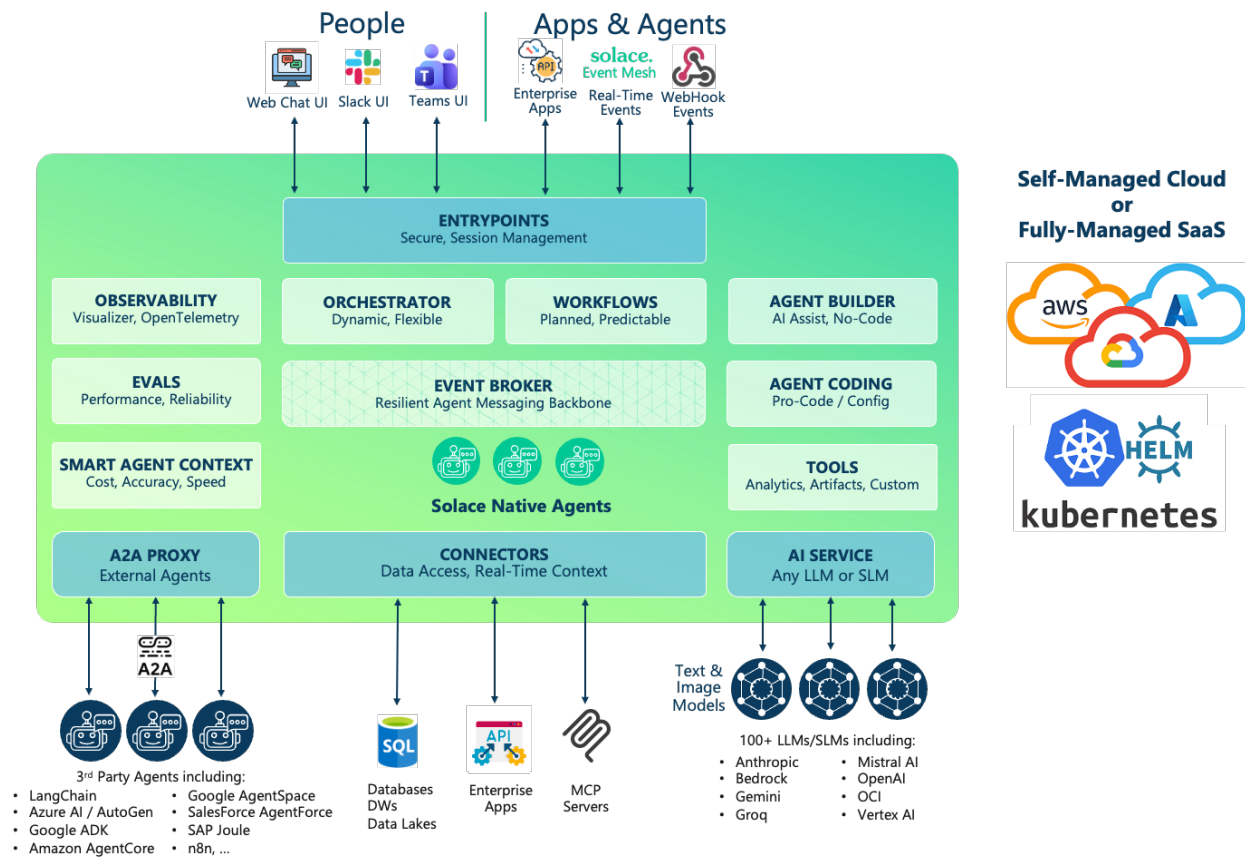
The agent development lifecycle is ultimately about creating the conditions for enterprise readiness. Ad hoc agent deployments can produce impressive demos. What they rarely produce is reliable and scalable adoption of agentic AI across an organization. This is similar to the situation where a small smart dev team might be able to build a working app, but enterprises have learned they need to adopt an SDLC to have broad, successful software deployment in production across the enterprise.

The ADLC provides the structure that bridges that gap. It applies lessons from software engineering methodology, distributed systems design, and organizational management to give enterprise architects a coherent approach to agent development. It acknowledges that building effective agentic systems is not purely a technical problem. It is a design problem, an organizational problem, and a governance problem all at once.

The organizations that take this seriously, that invest in structured agent hiring, careful onboarding, rigorous coaching and evaluation, deliberate supervision, thoughtful integration, and continuous improvement, are the ones that will turn the promise of agentic AI into durable enterprise value.

Appendix I: Solace Agent Mesh

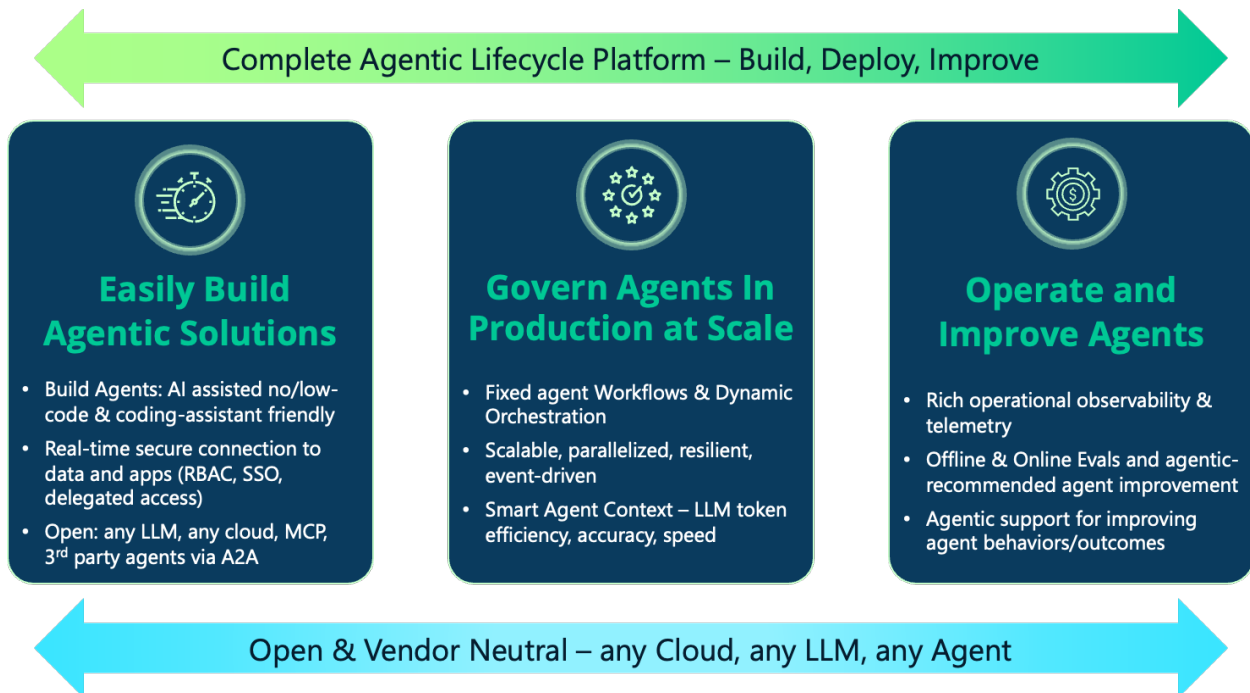
Solace Agent Mesh is the platform purpose-built to operationalize the ADLC end to end. It delivers all of the stages of a complete agentic lifecycle platform, on an open and vendor-neutral foundation that works with any cloud, any LLM, and any agent.



Solace Agent Mesh is built on the same asynchronous event-driven architecture that powers event-driven applications at many of the world's leading enterprises. It enables deterministic workflows and dynamic orchestration with agents communicating via asynchronous, decoupled messages — rather than synchronous API calls — inherently more resilient and scalable. Agents can run in parallel, recover from downstream failures gracefully, and scale independently without cascading bottlenecks.

Agent Mesh's smart agent context enables deliberate management of what actually enters the agent's context window. As agents operate over longer horizons and richer toolsets, raw data and unfiltered outputs crowd the context, degrading token efficiency, response accuracy, and latency. Smart agent context management keeps bulk data out of the LLM context while still making it accessible through artifact references, summarization, and structured retrieval. This reduces token costs, increases speed, and minimizes hallucinations.

Agent Mesh includes rich agent execution visualization, observability and evaluation capability to aid in easily understanding what agents are doing and why, and using feedback to close the loop into agentic-powered recommendations for improving agents.



Further Reading

Agentic AI systems are rapidly evolving from isolated task-specific components into collaborative networks distributed across environments, departments, and organizational boundaries. The architectural decisions you make today will determine whether these systems scale successfully or collapse under unaccounted-for limitations. We recommend reading <https://solace.com/lp/principles-agentic-design/>. This paper discusses:

- Why event-driven architecture provides a stronger foundation for multi-agent AI systems than point-to-point communication
- How to design AI agents like high-performing teams, with clear roles, boundaries, and coordination patterns
- How to manage context, memory, and information flow so agentic systems stay effective as they scale
- How to apply governance, security, and schema discipline to multi-agent systems in production
- How to choose the right first use cases and build toward enterprise-scale agentic AI with confidence

See how Solace Agent Mesh manages your AI workforce end to end: <https://solace.com/agent-mesh-demo/>